CSE 105:
Data Structures and Algorithms-I
(Part 2)

Instructor

Dr Md Monirul Islam

Heapsort

Heapsort

4	1	3	2	16	9	10	14	8	7

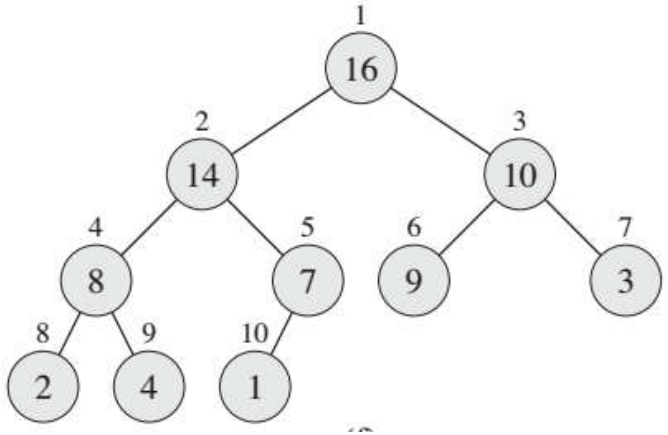
Heapsort

1	2	3	4	7	8	9	10	14	16

Heapsort

4	1	3	2	16	9	10	14	8	7

BuildMaxHeap



16	14	10	8	7	9	3	2	4	1

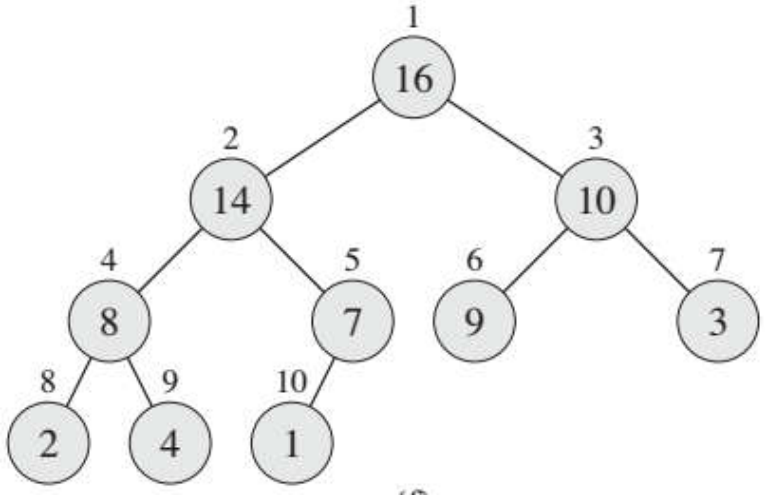
sort

1	2	3	4	7	8	9	10	14	16

Heapsort

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

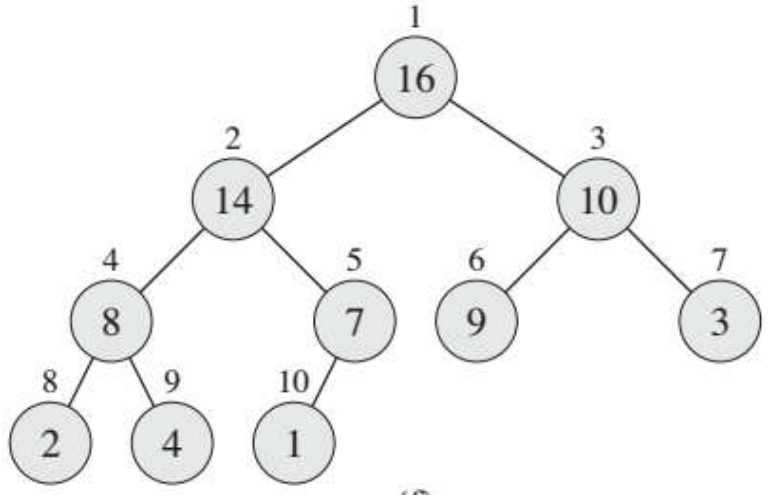
- Given **BuildHeap()**, a sorting algorithm can easily be constructed:
  - Maximum element is at $A[1]$
  - Discard by swapping with element at $A[n]$
    - Decrement heap_size[A]
    - $A[n]$ now contains correct value
  - Restore heap property at A[1] by calling **Heapify()**
  - Repeat, always swapping A[1] for A[heap_size(A)]



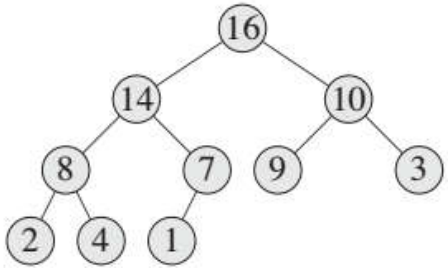
Heapsort

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

```
Heapsort(A) {
   BuildHeap(A);
   for (i = length(A) downto 2) {
         Swap(A[1], A[i]);
         heap_size(A) = heap_size(A) – 1;
         Heapify(A, 1);
   }
}
```

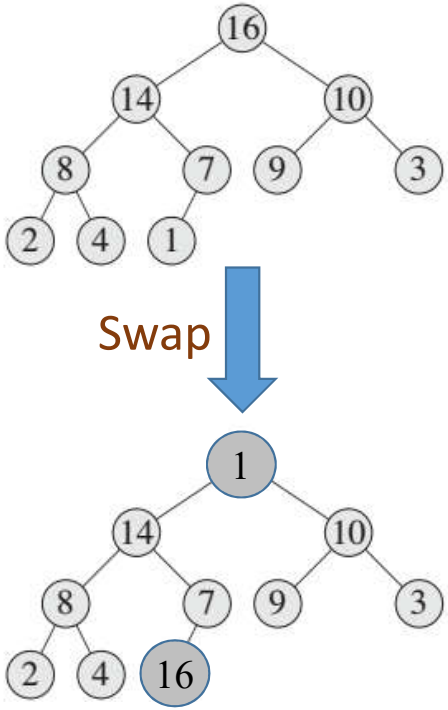


Heapsort Example



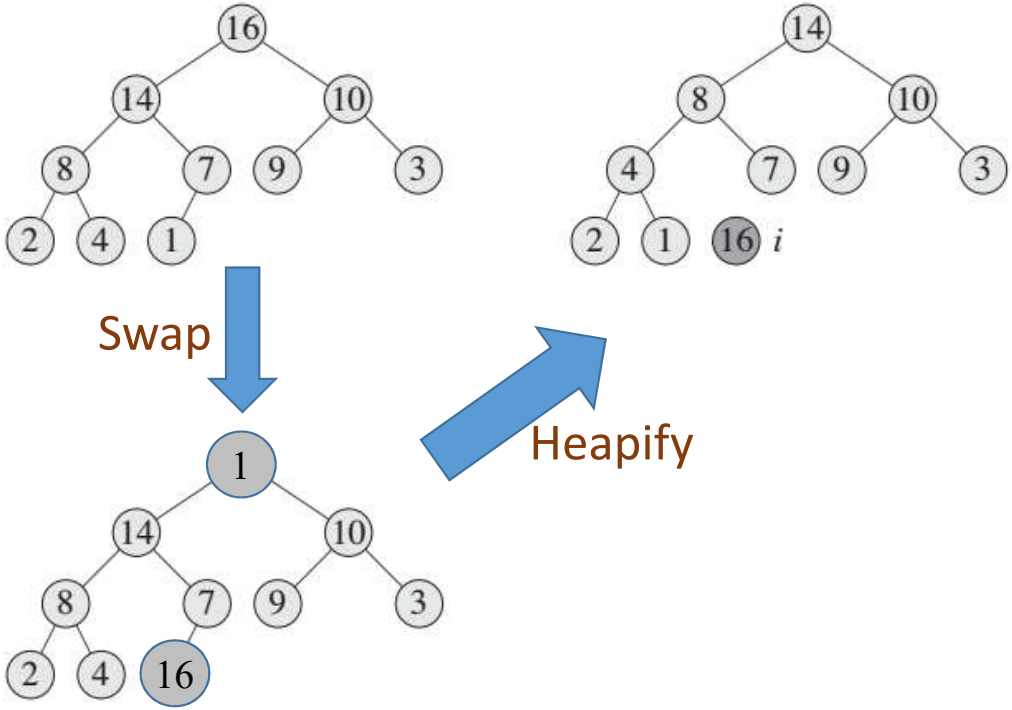
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Heapsort Example



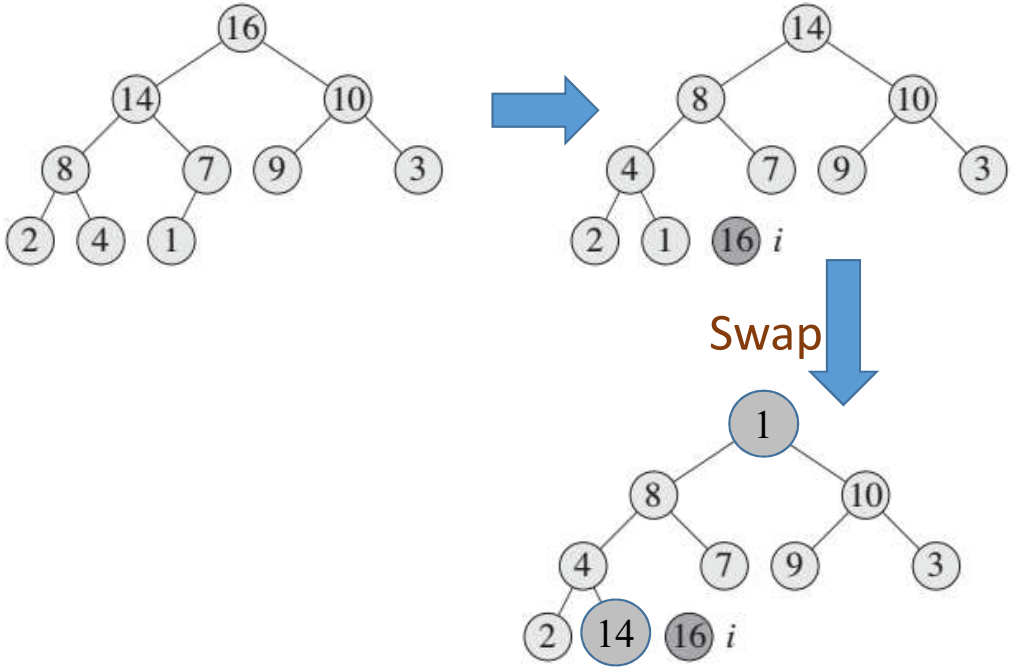
1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16

Heapsort Example



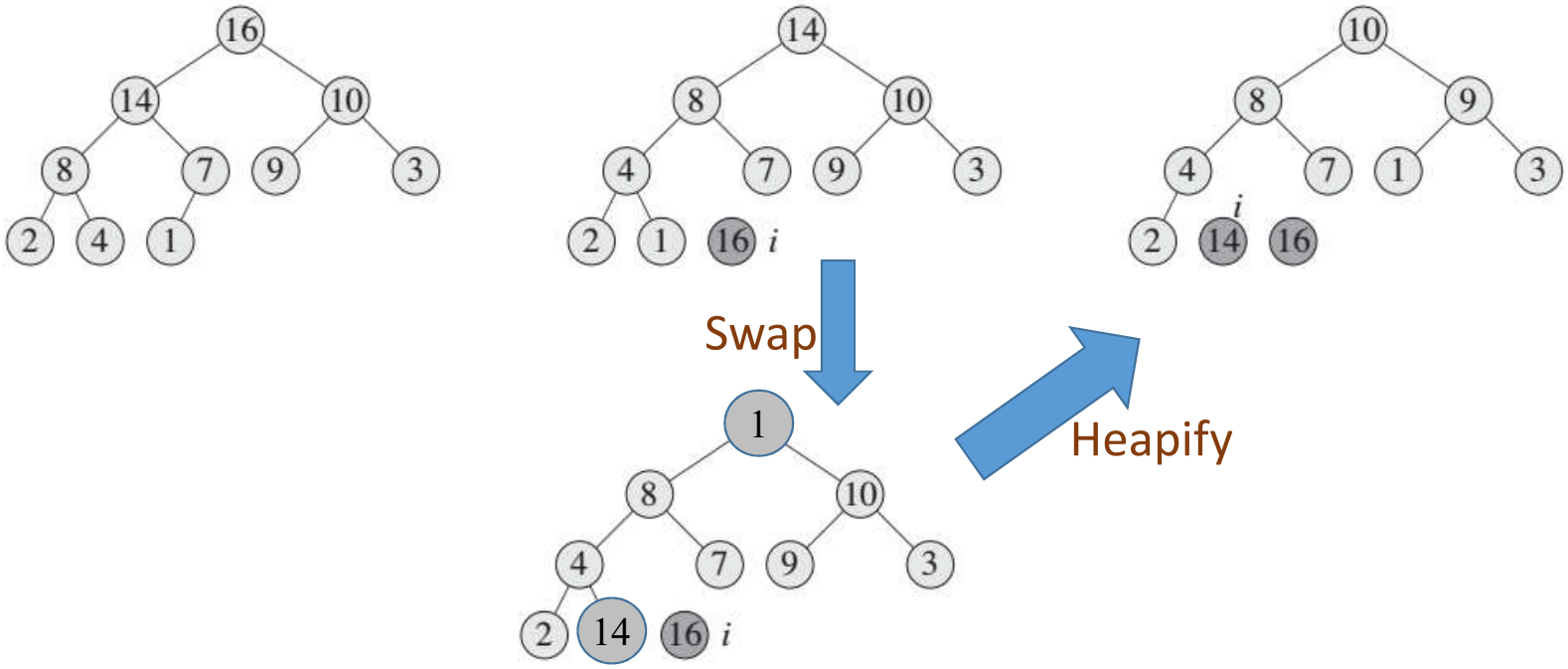
1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16

Heapsort Example



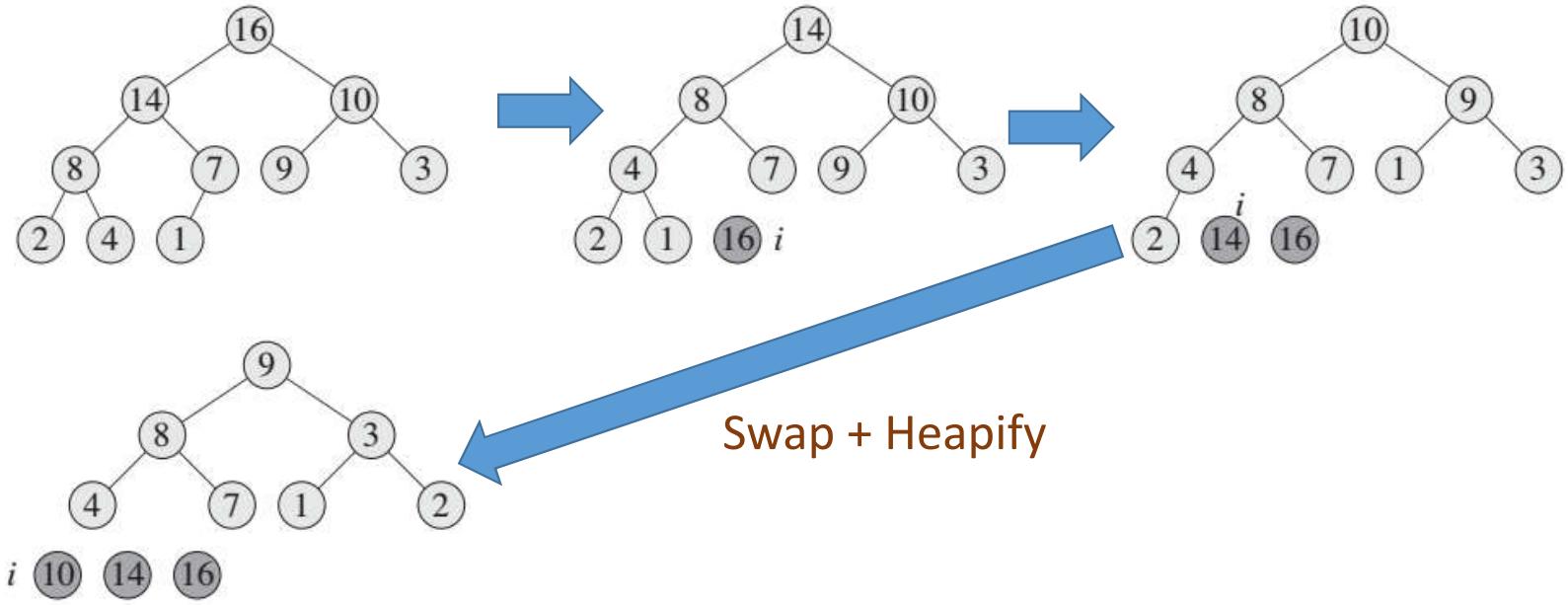
1	2	3	4	5	6	7	8	9	10
1	8	10	4	7	9	3	2	14	16

Heapsort Example



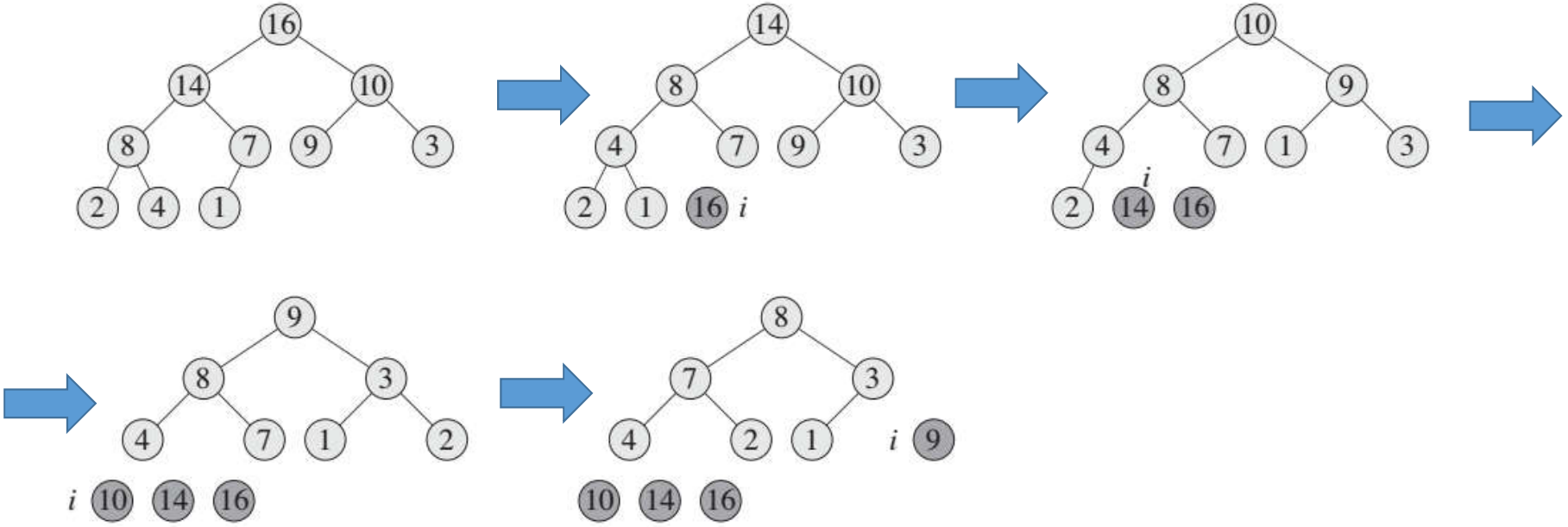
1	2	3	4	5	6	7	8	9	10
10	8	9	4	7	1	3	2	14	16

Heapsort Example



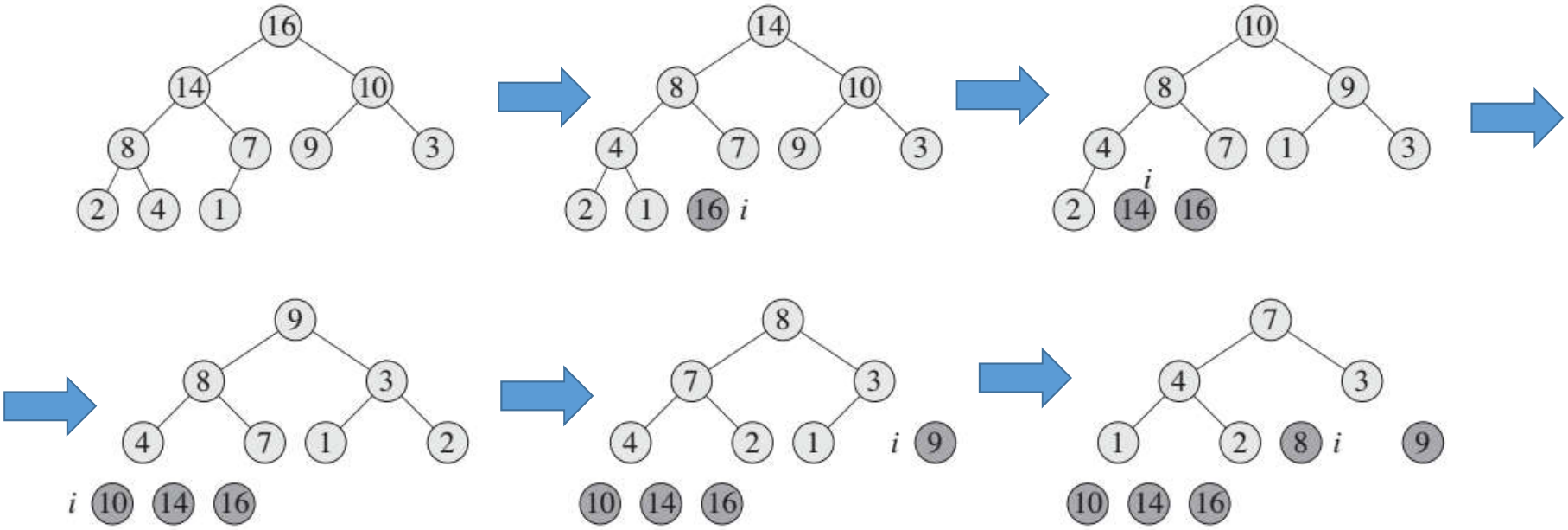
1	2	3	4	5	6	7	8	9	10
9	8	3	4	7	1	2	10	14	16

Heapsort Example



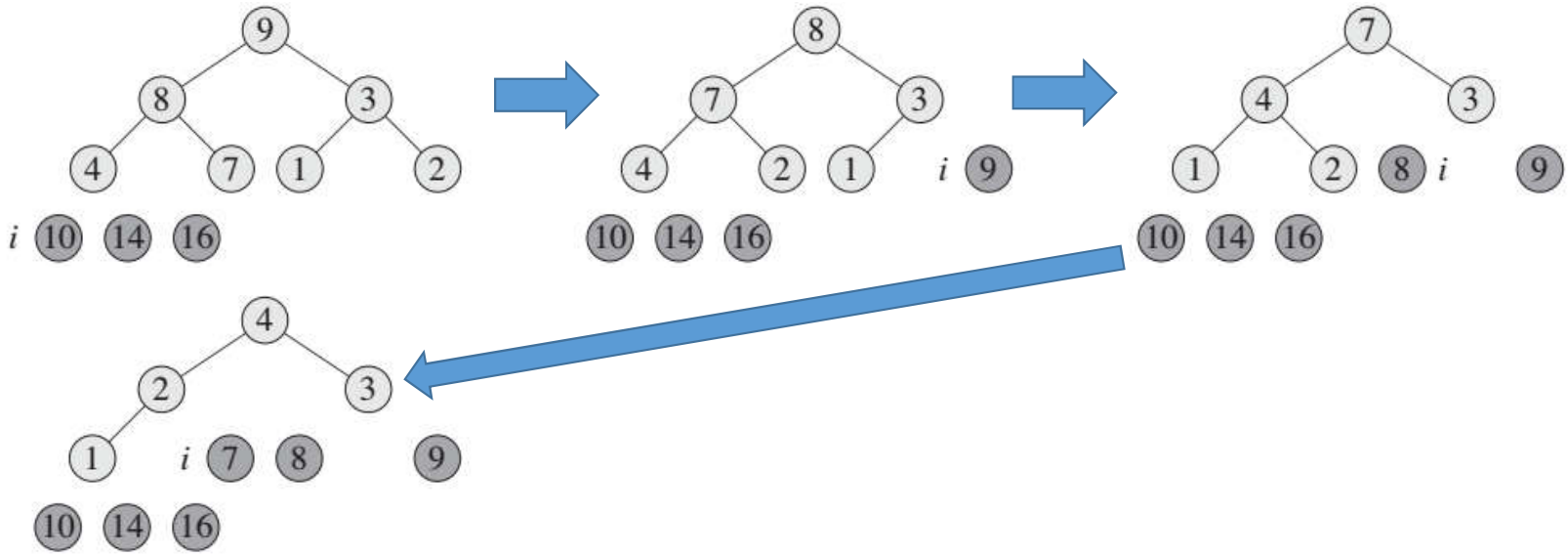
1	2	3	4	5	6	7	8	9	10
8	7	3	4	2	1	9	10	14	16

Heapsort Example



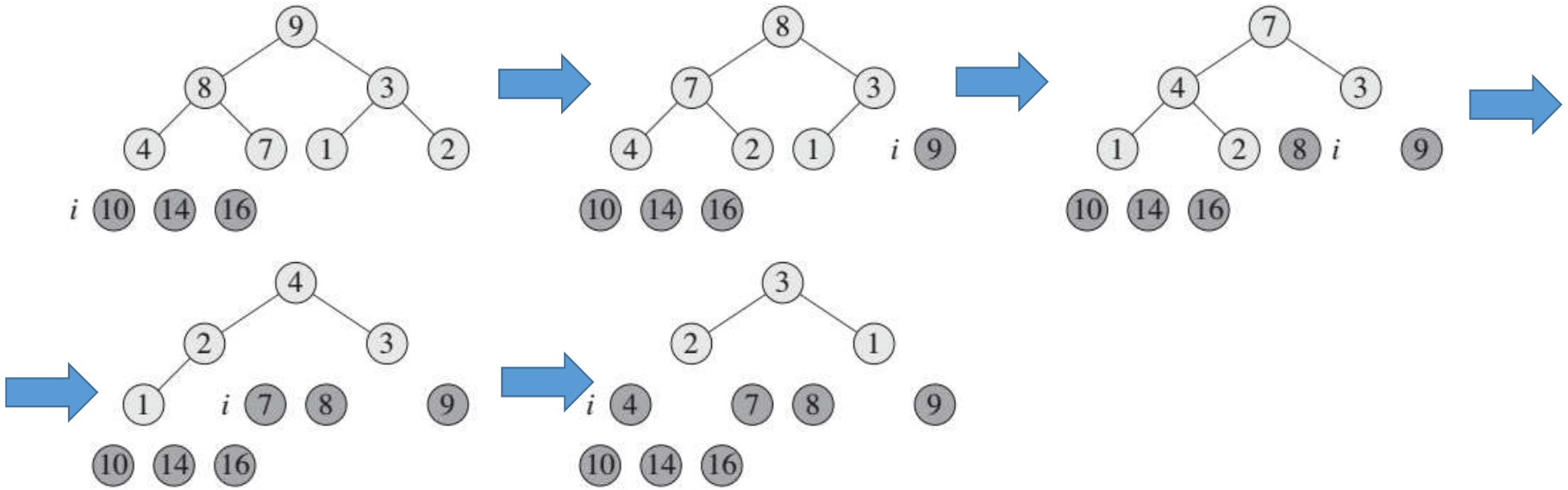
1	2	3	4	5	6	7	8	9	10
7	4	3	1	2	8	9	10	14	16

Heapsort Example



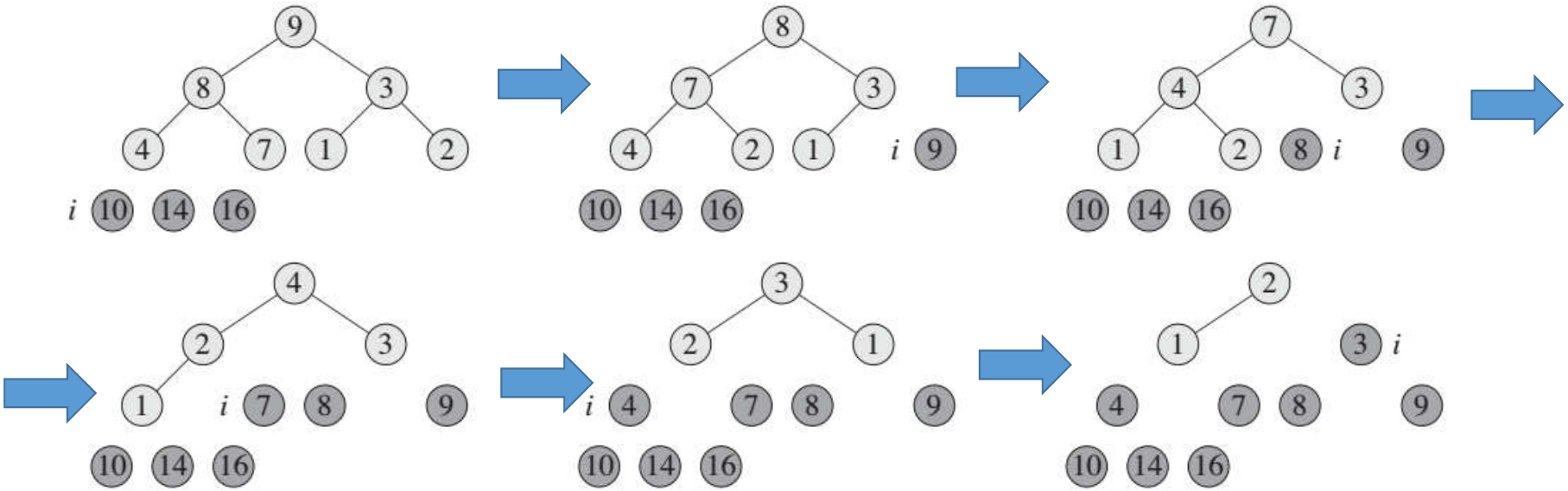
1	2	3	4	5	6	7	8	9	10
4	2	3	1	7	8	9	10	14	16

Heapsort Example



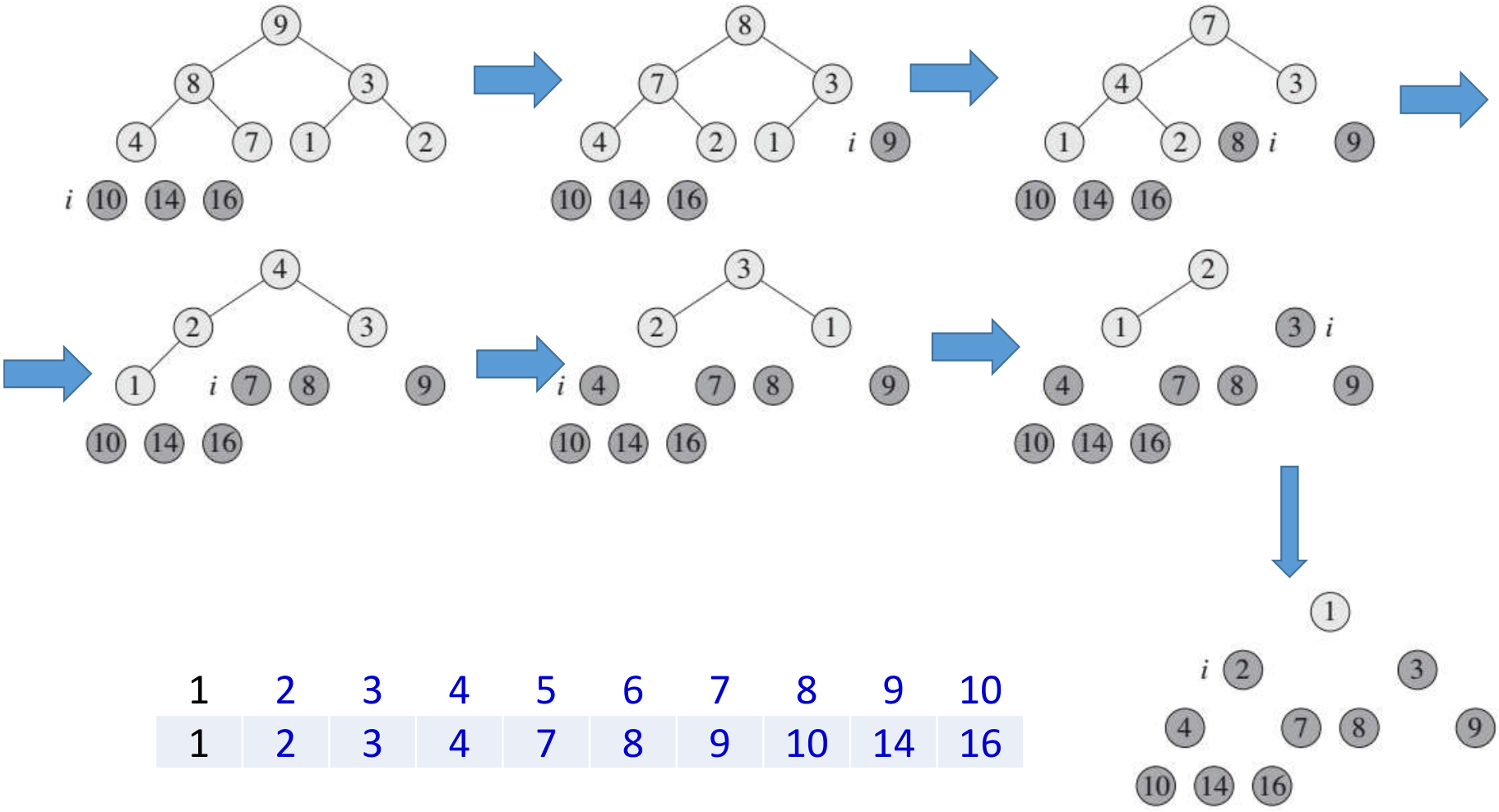
1	2	3	4	5	6	7	8	9	10
3	2	1	4	7	8	9	10	14	16

Heapsort Example



1	2	3	4	5	6	7	8	9	10
2	1	3	4	7	8	9	10	14	16

Heapsort Example



1	2	3	4	5	6	7	8	9	10
1	2	3	4	7	8	9	10	14	16

Analyzing Heapsort

```
Heapsort(A) {
  1. BuildHeap(A);
  2. for (i = length(A) downto 2) {
  3.       Swap(A[1], A[i]);
  4.       heap_size(A) =   heap_size(A) – 1;
  5.       Heapify(A, 1);
  }
}
```

- Line #2 loops for $n - 1$ time
- So, Heapify() at Line #5 is called (from this procedure) $n - 1$ times.

- The call to **BuildHeap()** takes O(n) time (Line #1)
- Each of the n - 1 calls to **Heapify()** takes O(lg n) time

Analyzing Heapsort

```
Heapsort(A) {
   1. BuildHeap(A);
   2. for (i = length(A) downto 2) {
   3.       Swap(A[1], A[i]);
   4.       heap_size(A) = heap_size(A) – 1;
   5.       Heapify(A, 1);
   }
}
```

- Line #2 loops for $n - 1$ time
- So, Heapify() at Line #5 is called (from this procedure) $n - 1$ times.

- The call to **BuildHeap()** takes $O(n)$ time (Line #1)
- Each of the n - 1 calls to **Heapify()** takes $O(\lg n)$ time
- Thus the total time taken by **HeapSort()**
  $= O(n) + (n - 1)\, O(\lg n)$
  $= O(n) + O(n \lg n)$
  $= O(n \lg n)$

Analyzing Heapsort

```
Heapsort(A) {
  1. BuildHeap(A);
  2. for (i = length(A) downto 2) {
  3.       Swap(A[1], A[i]);
  4.       heap_size(A) =   heap_size(A) – 1;
  5.       Heapify(A, 1);
  }
}
```

- Line #2 loops for $n - 1$ time
- So, Heapify() at Line #5 is called (from this procedure) $n - 1$ times.

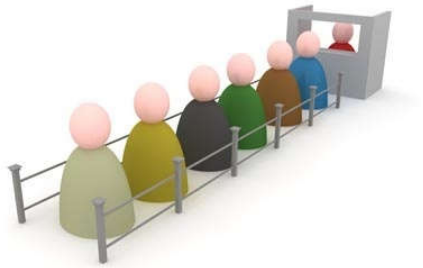
- The call to **BuildHeap()** takes $O(n)$ time (Line #1)
- Each of the n - 1 calls to **Heapify()** takes $O(\lg n)$ time
- Thus the total time taken by **HeapSort()**
  $= O(n) + (n - 1)\, O(\lg n)$
  $= O(n) + O(n \lg n)$
  $= O(n \lg n)$
- Heapsort is a nice algorithm, but in practice Quicksort (coming up) usually wins

Queues

FIFO: First in, First Out

Restricted form of list: Insert at one end, remove from the other.

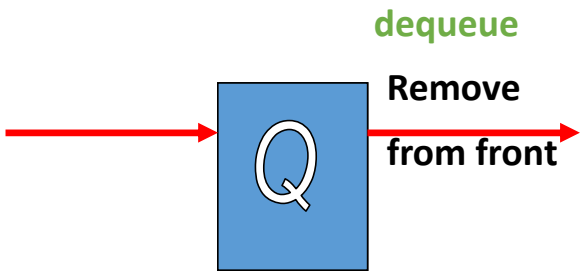
Notation:
- Insert: Enqueue
- Delete: Dequeue
- First element: Front
- Last element: Rear



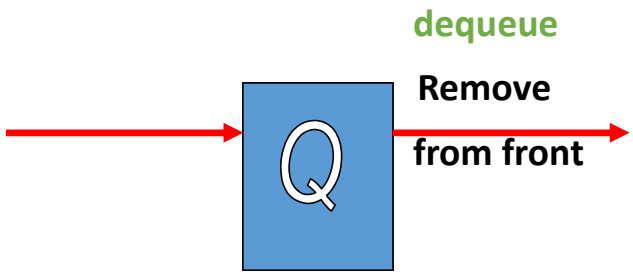
Priority Queues

A queue that is ordered according to some priority value

Standard Queue



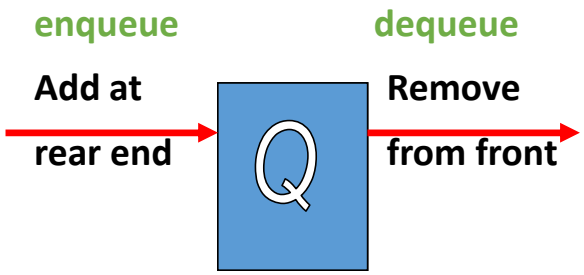
Priority Queue



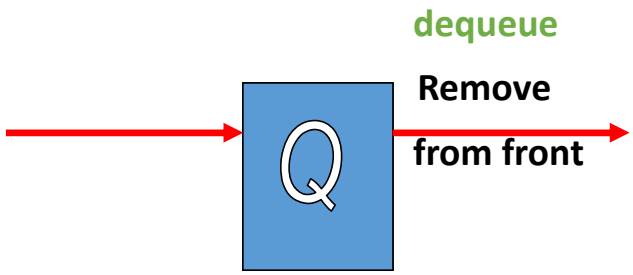
Priority Queues

A queue that is ordered according to some priority value

Standard Queue



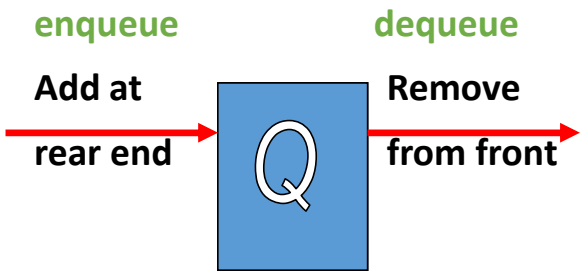
Priority Queue



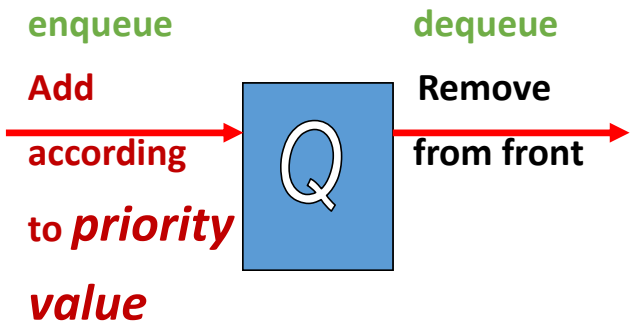
Priority Queues

A queue that is ordered according to some priority value

Standard Queue



Priority Queue



(MAX/MIN)-Priority Queues

- The heap data structure is incredibly useful for implementing *priority queues*
  - A data structure for maintaining a set *S* of elements, each with an associated value or *key*

- 2 classes
  - *max*-priority queue
  - *min*-priority queue

MAX-Priority Queues

- Applications:
  - we can use max-priority queues to schedule jobs on a shared computer.
    - The max-priority queue keeps track of the jobs to be performed and their relative priorities.
    - When a job is finished or interrupted, the scheduler selects the highest-priority job from among those pending.
    - The scheduler can add a new job to the queue at any time

MIN-Priority Queues

- Applications:
  - Simulating events
    - Events are simulated according to time of occurrence
    - The event with the next lowest time is to be generated first
    - One event can trigger multiple new events

Priority Queue Operations

Insert(*S, x*) – Inserts element *x* into set *S*, according to its priority

Maximum(*S*) – Returns, but does not remove, the element of *S* with the largest key

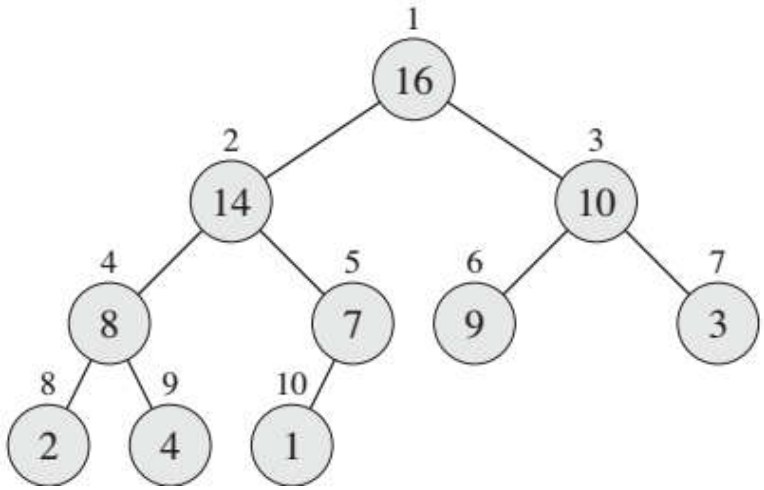
Extract-Max(*S*) – Removes and returns the element of *S* with the largest key

Increase-Key(*S, x, k*) – Increases the value of element *x*'s key to the new value *k*

How could we implement these operations using a heap?

Priority Queue Operations

```
HEAP-MAXIMUM(A)
1  return A[1]
```



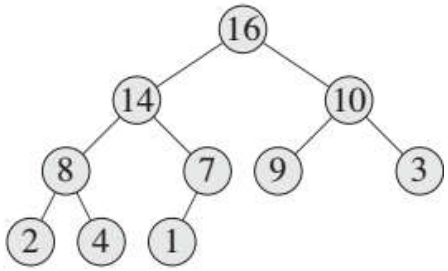
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2        error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```

Priority Queue Operations

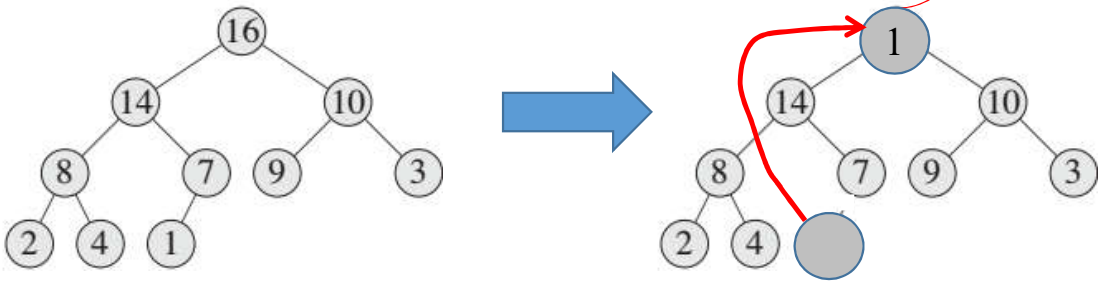
```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2        error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

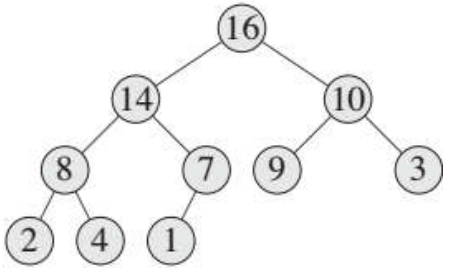
```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2        error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```

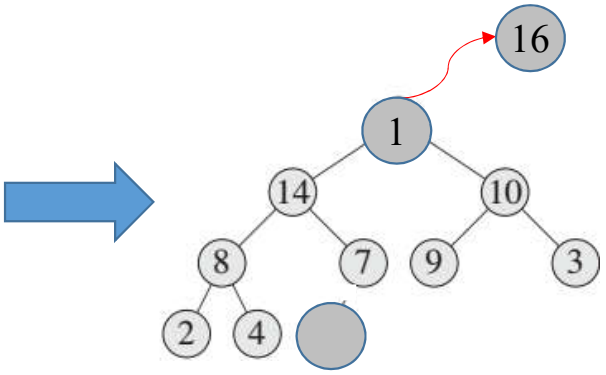


1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	

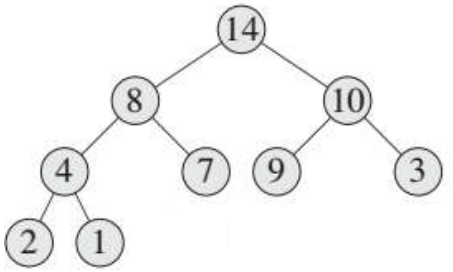
Priority Queue Operations

```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2        error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```





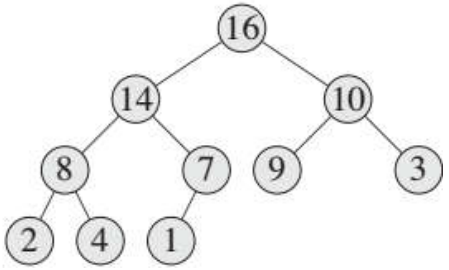
Max_Heapify

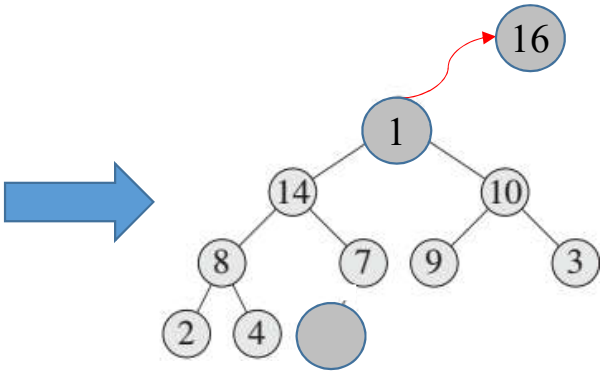


1	2	3	4	5	6	7	8	9
14	8	10	4	7	9	3	2	1

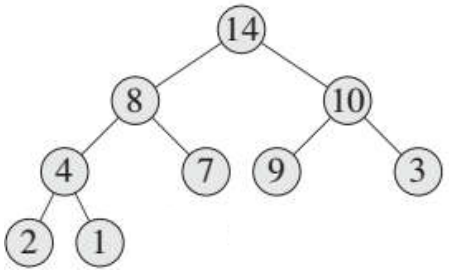
Priority Queue Operations

```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2        error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```





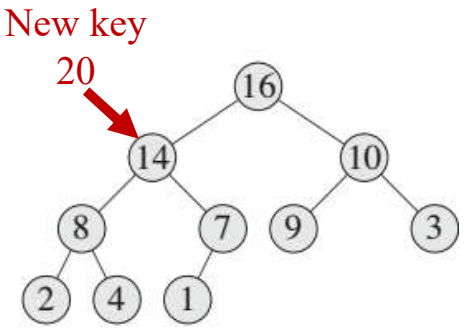
Max_Heapify



Complexity: O(logn)

1	2	3	4	5	6	7	8	9
14	8	10	4	7	9	3	2	1

Priority Queue Operations



Increase-Key(*S, x, k*) – Increases the value of element *x*'s key to the new value *k*

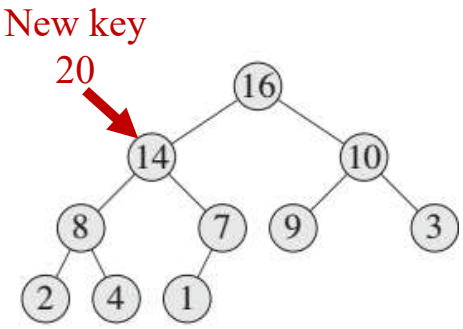
Heap relation

with parent may be violated

with children is maintained

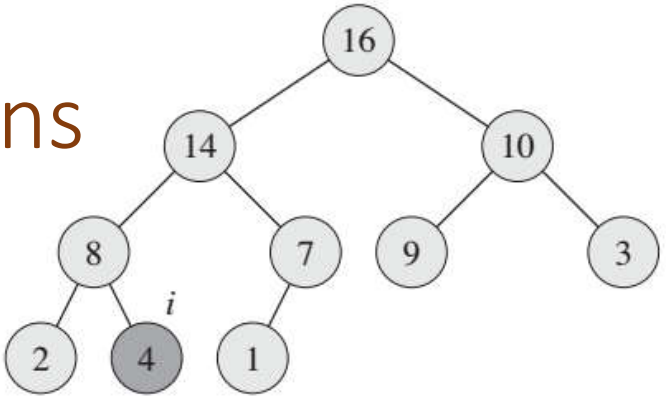
Priority Queue Operations

```
HEAP-INCREASE-KEY(A, i, key)
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```



Priority Queue Operations

```
HEAP-INCREASE-KEY(A, i, key)
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```

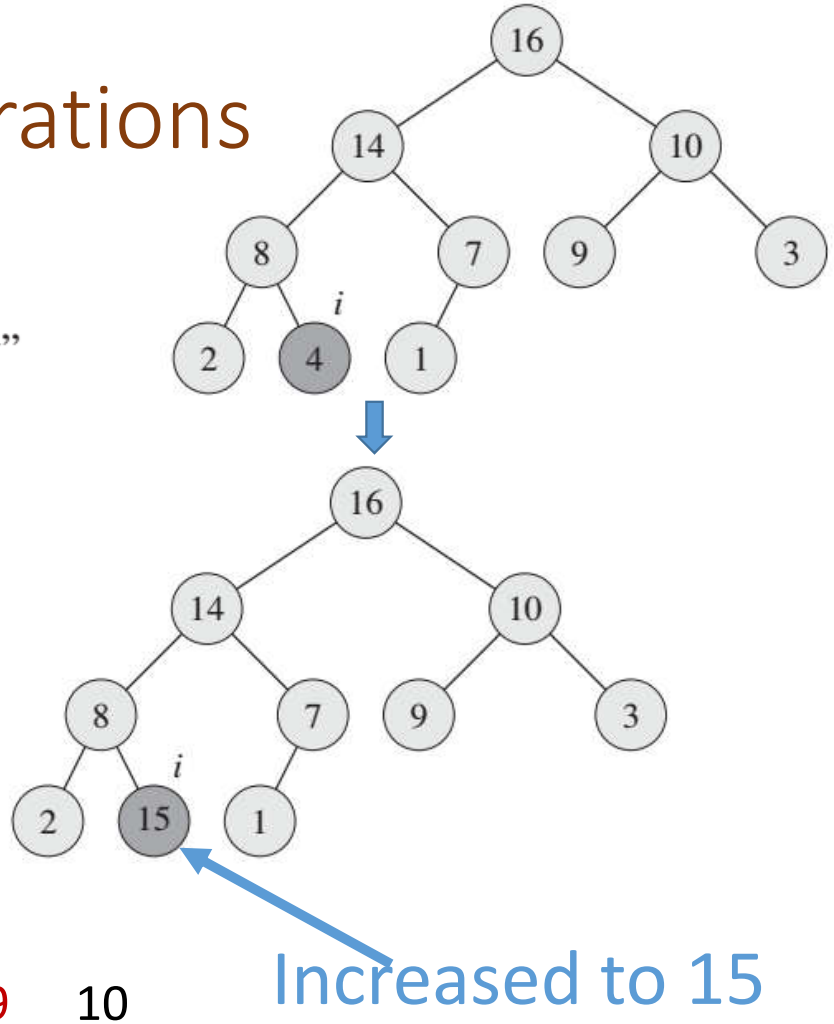


1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

HEAP-INCREASE-KEY(A, i, key)

```
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```

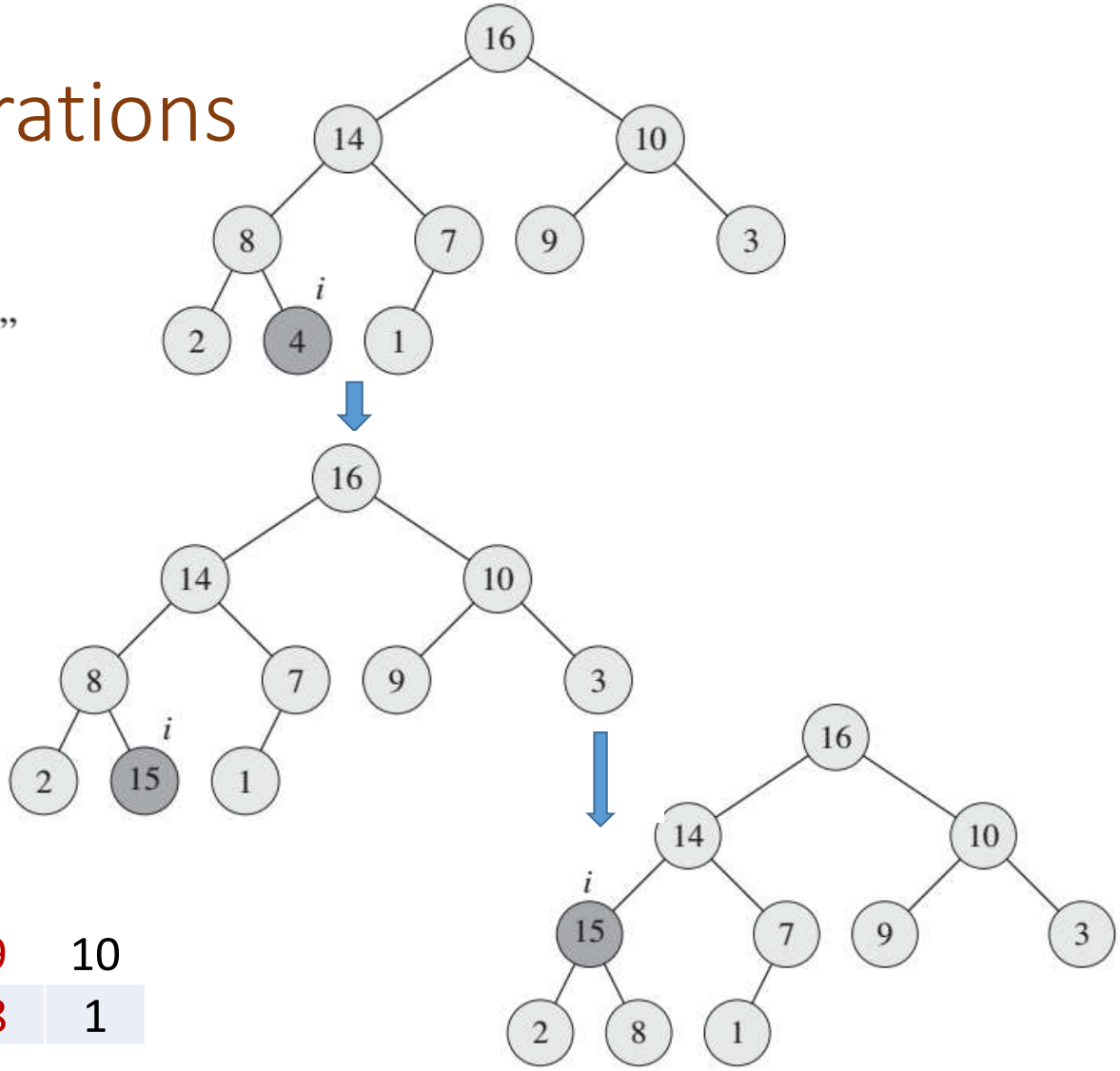


1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	15	1

Priority Queue Operations

HEAP-INCREASE-KEY(A, i, key)

```
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```

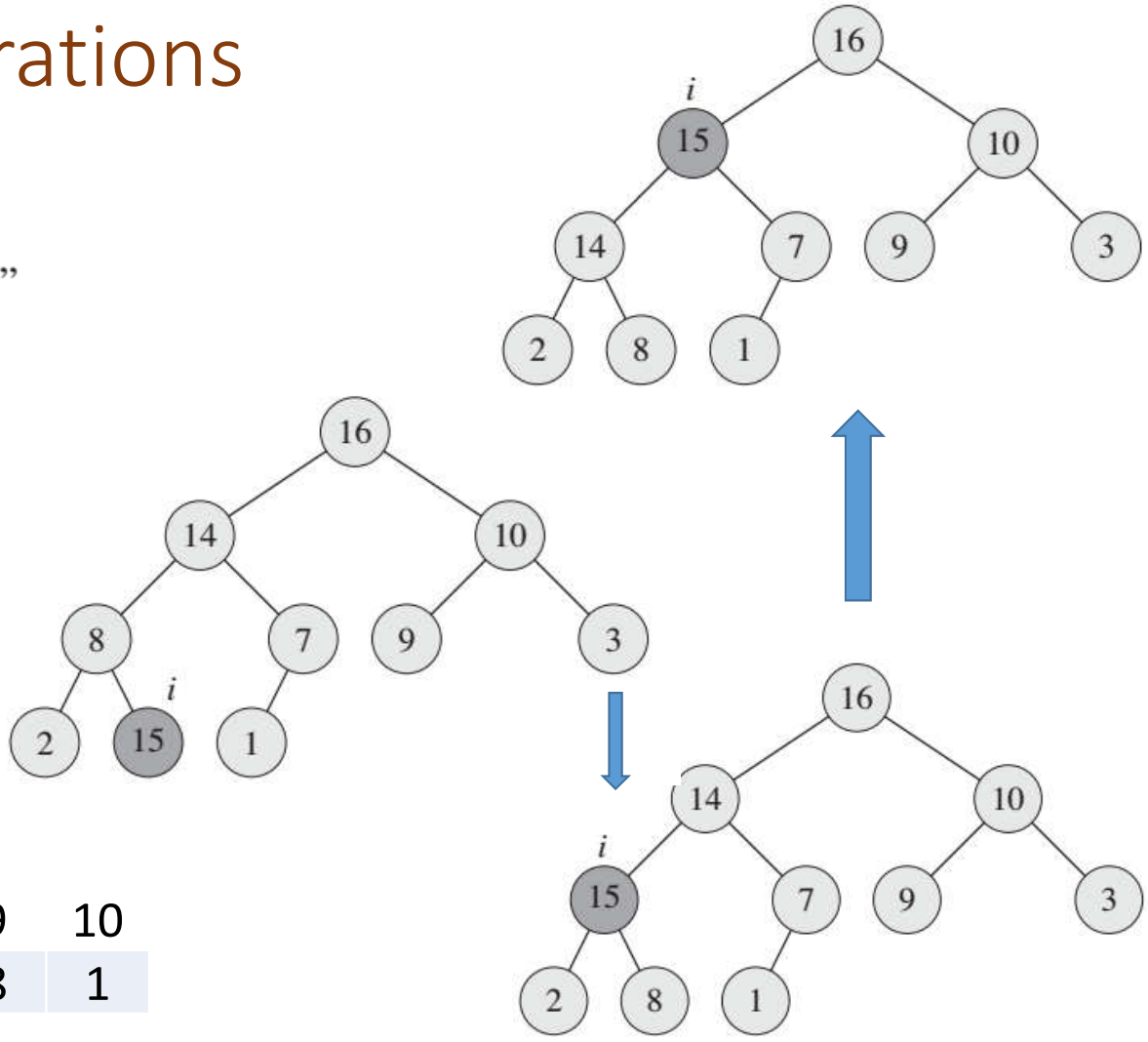


1	2	3	4	5	6	7	8	9	10
16	14	10	15	7	9	3	2	8	1

Priority Queue Operations

HEAP-INCREASE-KEY(A, i, key)

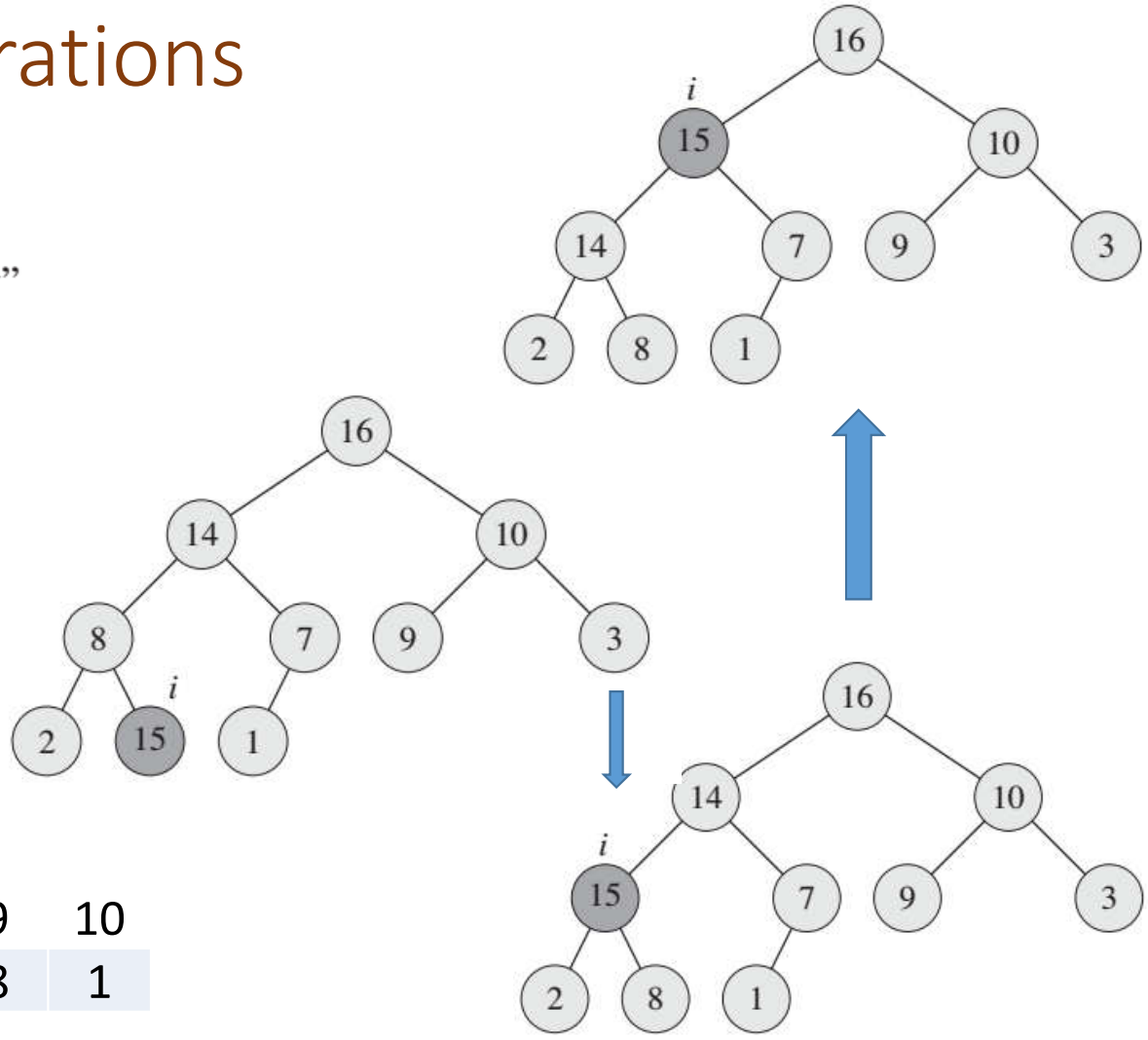
```
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```



1	2	3	4	5	6	7	8	9	10
16	15	10	14	7	9	3	2	8	1

Priority Queue Operations

```
HEAP-INCREASE-KEY(A, i, key)
1  if key < A[i]
2        error "new key is smaller than current key"
3  A[i] = key
4  while i > 1 and A[PARENT(i)] < A[i]
5        exchange A[i] with A[PARENT(i)]
6        i = PARENT(i)
```



Complexity: O(logn)

1	2	3	4	5	6	7	8	9	10
16	15	10	14	7	9	3	2	8	1

Priority Queue Operations

```
MAX-HEAP-INSERT(A, key)
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size] = −∞
3  HEAP-INCREASE-KEY(A, A.heap-size, key)
```

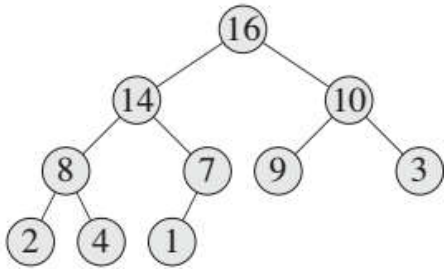
Insert(S, x) – Inserts element x into set S, according to its priority

1. Create a **new leaf** with -∞
2. Then increase its value to *key*

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$

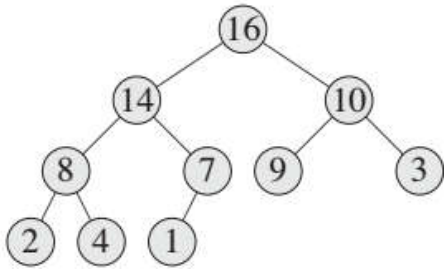


1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$



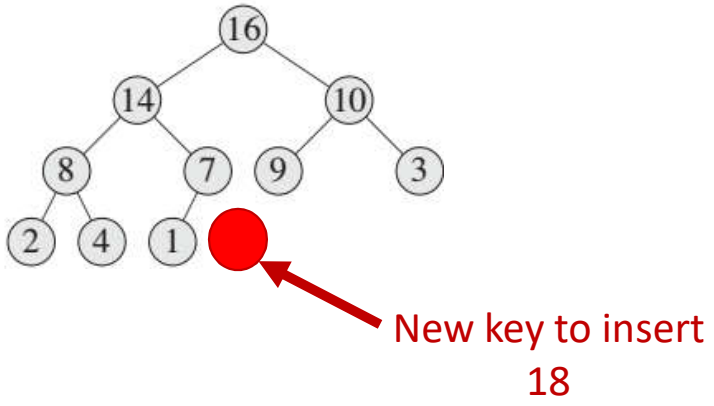
New key to insert
18

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$

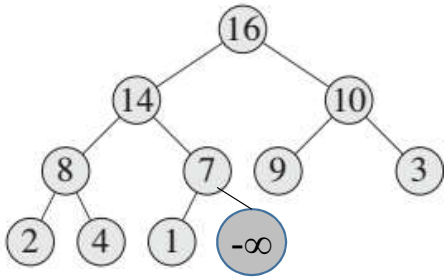


1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$

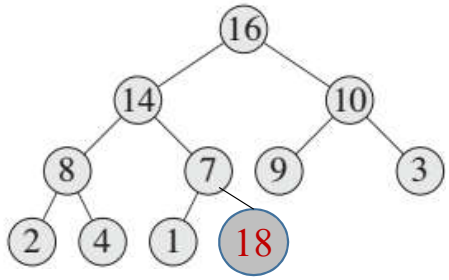


1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-∞

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$

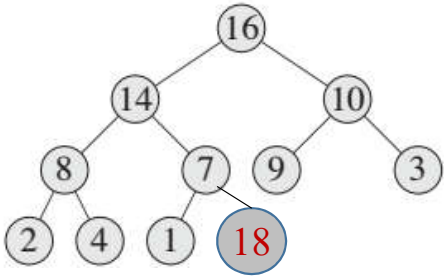


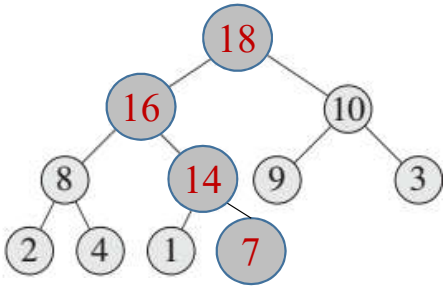
1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	18

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$



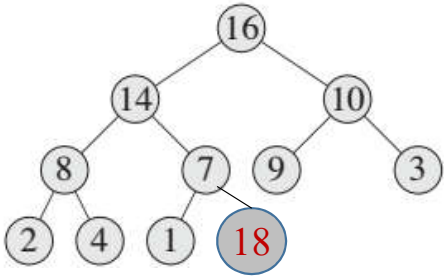


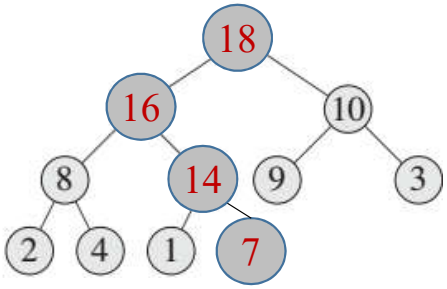
1	2	3	4	5	6	7	8	9	10	11
18	16	10	8	14	9	3	2	4	1	7

Priority Queue Operations

MAX-HEAP-INSERT(A, key)

1 $A.heap\text{-}size = A.heap\text{-}size + 1$
2 $A[A.heap\text{-}size] = -\infty$
3 HEAP-INCREASE-KEY$(A, A.heap\text{-}size, key)$





Complexity: O(logn)

1	2	3	4	5	6	7	8	9	10	11
18	16	10	8	14	9	3	2	4	1	7